

以下是基于你简历内容整理的**模拟面试问题**，涵盖**项目细节、技术深度、设计决策、场景题、基础原理**等方向，面试官会重点“拷打”这些点，建议你提前准备好回答：

一、项目相关（深挖细节）

镜海视频平台

1. 突破讯飞星火QPS限制的方案

- “多个账号轮流使用”具体是怎么实现的？账号池如何管理？
- 修改凭证状态时如何保证线程安全？
- 如果所有账号都达到QPS上限怎么办？有没有降级方案？

2. 分片上传 + 断点续传

- 分片大小如何确定？上传失败后如何记录已上传的分片？
- 断点续传的“断点”存在哪里？服务端还是客户端？
- 如何保证分片合并后文件的完整性？

3. 服务端中转实现私聊

- 为什么不用WebSocket直接点对点？
- 消息离线如何处理？
- 私聊消息的顺序性如何保证？

4. 创作辅助 Agent

- RAG检索用的是哪些文档？向量库是什么？
- 本地知识库如何构建和更新？
- 生成的标题/简介如何评估质量？

5. 双JWT实现无感刷新

- 双JWT是哪两个？它们的关系是什么？
- 刷新Token过期了怎么办？
- 如何防止刷新Token被滥用？

6. 数据同步 (ES + MySQL)

- 为什么选择XXL-JOB + RocketMQ + OpenFeign？
- 数据同步延迟问题如何监控？
- 如果消息重复消费怎么办？

7. 流量合并请求

- 阈值如何设定？动态调整还是固定？
 - 合并后如何区分不同请求的响应？
 - 超时机制如何设计？超时后怎么处理？
-

蜂巢玩聚互动网络

8. 乐观锁实现库存扣减

- 高并发下乐观锁重试次数多怎么办？
- 有没有考虑过Redis + Lua + 悲观锁？
- 如果库存扣减失败，如何保证不超卖？

9. Redis + Lua脚本实现预检

- Lua脚本里写了哪些逻辑？
- 预检失败后用户如何知道原因？
- 预检与真正扣库存之间状态如何同步？

10. 逻辑过期缓存

- “逻辑过期”是怎么实现的？
- 缓存重建时如果多个请求同时进来怎么办？
- 为什么要用逻辑过期而不是物理过期？

二、技术基础（深挖原理）

Java & JUC

- 11. **CAS**：ABA问题如何解决？
- 12. **AQS**：讲一下ReentrantLock的实现原理
- 13. **线程池**：核心参数如何设置？拒绝策略有哪些？
- 14. **ConcurrentHashMap**：JDK 1.7 和 1.8 的实现区别？

JVM

- 15. **类加载机制**：双亲委派模型为什么要这样设计？
- 16. **垃圾回收**：G1 和 CMS 的区别？
- 17. **对象创建**：对象在内存中是如何布局的？

Spring

- 18. **Bean生命周期**：讲一下从加载到销毁的全过程
- 19. **循环依赖**：Spring 如何解决？为什么需要三级缓存？
- 20. **AOP**：动态代理与CGLIB的区别？

MySQL

- 21. **索引**：B+树索引与哈希索引的区别？
- 22. **事务**：MVCC 是如何实现的？
- 23. **SQL优化**：你是如何定位慢SQL的？举一个真实案例

Redis

- 24. **分布式锁**：如何保证锁的释放？
- 25. **持久化**：RDB 和 AOF 的优缺点？
- 26. **哨兵机制**：哨兵集群是如何选举Leader的？

RocketMQ

- 27. **消息确认机制**：ACK 机制是怎样的？
- 28. **死信队列**：什么情况会进入死信队列？
- 29. **顺序消息**：如何保证全局有序？

三、场景设计题（考察工程能力）

- 30. **弹幕系统设计**
 - 如何支持高并发弹幕推送？
 - 弹幕存储用什么数据库？如何分表？
- 31. **秒杀系统优化**
 - 如果瞬时流量 10w/s，你会怎么设计？
 - 如何防止脚本刷单？
- 32. **分布式ID生成**
 - 雪花算法有什么问题？如何解决时钟回拨？
- 33. **限流算法**
 - 漏桶 vs 令牌桶，分别适用什么场景？

四、非技术问题（软技能 & 职业素养）

- 34. 你在这个项目中遇到的最大技术难题是什么？怎么解决的？
- 35. 如果让你重新设计这个项目，你会改进哪里？
- 36. 你如何保证代码质量？
- 37. 你平时怎么学习新技术？
- 38. 你希望从实习中获得什么？

如果你希望我**逐个问题给出参考答案**，或者**模拟一场完整的面试对话**，也可以告诉我，我可以继续帮你准备。

没问题，我会**逐个问题给出“标准参考答案”**（可以直接背、面试时说出来）+ **“大白话理解”**（帮你搞懂背后的原理，方便你灵活变通）。

你的基础不扎实，但面试是可以短期准备的。下面这些内容，**先理解** → **再记关键词** → **最后用自己的话说出来**。不要死记硬背。

一、镜海视频平台（项目深挖）

1. 突破讯飞星火QPS限制的方案

标准参考答案

“我们通过**账号池**来突破QPS限制。事先准备多个合法账号，每个账号的QPS上限是2。
请求进来时，从账号池中**获取一个当前未达到QPS上限的账号**，使用完后更新该账号的调用计数。
账号池的状态用**Redis + Lua脚本**保证原子性。
如果所有账号都达到上限，请求会**阻塞或快速失败**，并记录告警，便于动态扩容账号池。”

大白话理解

- 讯飞星火免费版1秒钟只能处理2个请求（QPS=2）。
- 我们搞了10个账号，每个都能处理2个，总共就是20。
- 用一个“池子”存账号，每次来请求就找一个有空闲的账号去调。
- 用Redis记录每个账号当前在1秒内已经调了多少次，Lua脚本保证不会同时有两个请求抢同一个账号。
- 如果所有账号都在忙，那就让用户等一下或者直接提示“太忙了”。

2. 分片上传 + 断点续传

标准参考答案

“分片上传：将大文件按固定大小（如5MB）切分成多个分片，每个分片单独上传。
上传前从服务端获取一个**上传ID**，每个分片携带该ID和分片编号。
服务端收到分片后先存为临时文件，并记录已上传的分片列表到Redis。
断点续传：客户端上传前先询问服务端已上传的分片编号，然后**跳过已上传的分片**，继续上传剩余分片。
所有分片上传完成后，服务端按顺序合并分片，并校验文件完整性（如MD5）。”

大白话理解

- 一个大视频好比一本很厚的书。
- 分片上传：把书拆成一页一页寄过去。
- 断点续传：寄到第50页时网断了，重新连上后问对方“你收到哪一页了？”对方说“第49页”，你就从第50页继续寄，不用从头来。
- 最后把所有页按顺序粘成一本书。

3. 服务端中转实现私聊

标准参考答案

“用户A发给用户B的消息，先发到**服务端**，服务端再转发给B。
这样做的好处：

1. **离线消息**：如果B不在线，服务端可以先存到数据库，等B上线再推送。
2. **消息顺序**：服务端可以给消息加全局递增序号，保证不乱序。
3. **安全性**：可以过滤敏感词、记录聊天日志。
为什么不用点对点？因为浏览器/App之间无法直接建立连接，必须经过服务器。”

大白话理解

- 你给朋友发微信，不是直接飞到朋友手机上，而是先到腾讯服务器，服务器再发给朋友。
- 朋友关机了，服务器先存着，等他一开机就发给他。
- 如果让你直接连朋友手机，第一你找不到他，第二不安全。

4. 创作辅助 Agent (RAG + 本地知识库)

标准参考答案

“我们使用LangChain4j搭建Agent。

RAG检索：用户上传视频后，从视频中提取音频转文字，结合**本地知识库**（平台已有的优秀标题/简介范例）和**向量数据库**（如Elasticsearch或Milvus）做相似度检索。

本地知识库定期更新：从历史高播放量视频中挖掘优质描述，人工审核后入库。

生成质量评估：通过A/B测试，看使用Agent的创作者发布后的视频平均播放量是否提升。”

大白话理解

- 你上传了一个打篮球的视频，Agent会去“优秀标题库”里找类似视频的标题，比如“最帅绝杀”之类的。
- 还会看视频内容（语音转文字），如果你说“这个三分太牛了”，它可能会生成“三分神准！不看后悔”。
- 好不好用？对比用了这个功能的up主，播放量涨没涨。

5. 双JWT实现无感刷新

标准参考答案

“双JWT：一个**Access Token**（短时效，例如30分钟），一个**Refresh Token**（长时效，例如7天）。用户登录时下发两个Token。

每次请求携带Access Token，服务端验证其有效性。

当Access Token过期时，客户端携带Refresh Token去请求**刷新接口**。

服务端验证Refresh Token有效后，颁发新的Access Token（和新的Refresh Token，滚动刷新）。

如果Refresh Token也过期，则要求重新登录。

这样既保证了安全性（Access Token短），又无需用户频繁登录。”

大白话理解

- Access Token像“临时工牌”，30分钟就失效，丢了也不怕。
- Refresh Token像“身份证”，7天有效，放在更安全的地方（比如HttpOnly Cookie）。
- 工牌过期了，你拿身份证去换一个工牌，不用重新面试（登录）。
- 身份证也过期了，那就只能重新登录。

6. 数据同步 (ES + MySQL)

标准参考答案

“我们使用**canal**或**消息队列**实现最终一致性。

具体方案：MySQL变更时，通过**Binlog**触发消息发送到RocketMQ，消费者消费消息后更新ES。

之所以用XXL-JOB + Redis + RocketMQ + OpenFeign，是为了**补偿机制**：定时任务扫描Redis中未成功同步的数据，重新发送到MQ。

如果消息重复消费，我们在ES中通过**幂等设计**（使用业务ID + 版本号）避免重复更新。

同步延迟通过**监控**：对比MySQL和ES的数据时间戳，超过阈值告警。”

大白话理解

- MySQL是主仓库，ES是搜索引擎的索引。
- 主仓库里商品价格改了，要通知ES也改，不然用户搜到的还是老价格。
- 正常情况：MySQL一变，自动发消息给ES。
- 如果消息丢了（比如网络抖了一下），定时任务（XXL-JOB）会检查哪些没同步成功，重新发一遍。
- 重复发也没事，因为ES里会判断“如果这条数据的版本号比现有的旧，就不更新”。

7. 流量合并请求

标准参考答案

“我们使用**请求合并**（Request Coalescing）降低下游压力。

思路：将一段时间内（如10ms）对**相同资源**的多个请求合并成一个批量请求。

实现：

- 请求进来时，放入一个**阻塞队列**，并等待一个短窗口（阈值）。
 - 窗口结束后，一次性从队列中取出所有请求，组装成批量参数调用下游。
 - 响应回来后，根据每个请求的标识拆分结果，分别返回给各个客户端。
 - 超时机制：如果某个请求等待超过100ms，直接返回降级结果，避免长时间阻塞。
- 阈值动态调整：根据当前QPS和平均响应时间自适应。”

大白话理解

- 很多人同时查“商品A的价格”。
- 不要每人单独去数据库查一次，而是**攒一波**，比如10ms内的所有请求，一次去数据库查“A、B、C”多个商品。
- 数据库返回一个列表，再拆开分给每个人。
- 如果攒的时间太长（比如100ms还没凑够一波），就先把手上的几个查了，别让用户等死。

二、蜂巢玩聚互动网络（项目深挖）

8. 乐观锁实现库存扣减

标准参考答案

“我们使用**乐观锁**：

```
update product set stock = stock - 1, version = version + 1 where id = ? and version = old_version and stock > 0
```

如果更新行数为0，说明版本号变了（被别人改过），则重试。

高并发下重试次数过多怎么办？

- 限制最大重试次数（如3次），超过则失败。
- 或者改用Redis预扣减 + 异步落库。
乐观锁适合冲突不严重的场景。秒杀冲突极高时，会改用Redis + Lua或悲观锁。”

大白话理解

- 乐观锁假设“不会打架”。
- 你拿着版本号1去改，改之前看一眼版本号还是1吗？是就改，同时版本号变成2。
- 如果你犹豫的时候别人已经改成2了，你就改不了，只能重试。
- 抢茅台那种几万人抢一瓶，重试会累死，就不适合乐观锁。

9. Redis + Lua脚本实现预检

标准参考答案

“预检脚本在Redis中原子执行，检查：

1. 用户是否已抢过
2. 库存是否充足
3. 是否在活动时间内

全部通过则返回成功，并**预扣库存**（记录到Redis）。

预检成功后，会发送MQ消息，由消费者真正扣减数据库库存。

如果预检失败，直接返回具体原因（如“已抢过”“库存不足”）。

预检与真实扣库存之间通过**事务消息**或**定时对账**保证最终一致。”

大白话理解

- 你点“秒杀”，先不碰数据库（太慢），而是在Redis里用Lua脚本快速检查：
 - 你买过没？
 - 还有货吗？
- 检查通过，先在Redis里标记“你占了一件”，然后发个消息告诉数据库慢慢扣。
- 如果数据库扣失败了（比如突然缺货），后面再补偿。

10. 逻辑过期缓存

标准参考答案

“逻辑过期：不设置Redis的TTL，而是存储一个**过期时间戳**字段。

查询时判断 `当前时间 > 过期时间`，如果过期则**异步重建缓存**，同时**先返回旧数据**。

重建时使用**分布式锁**（如Redisson）防止多个请求同时重建。

其他请求在重建期间仍然读取旧数据，避免了缓存雪崩和穿透。

物理过期：直接删除key，重建期间所有请求打到数据库，压力大。”

大白话理解

- 正常缓存：到点就删，然后所有人发现没了，一起去查数据库（数据库可能会被压垮）。
- 逻辑过期：到点了不删，只是标记“旧了”。

- 有人来了，发现旧了，就去更新缓存，但更新期间仍然把旧数据返回给用户。
- 用户不卡顿，数据库压力也小。

三、技术基础 (Java & JUC)

11. CAS的ABA问题

标准参考答案

“ABA问题：线程1把变量从A改成B又改回A，线程2看到还是A，认为没变过，但实际上已经变过了。
解决：使用`AtomicStampedReference`或`AtomicMarkableReference`，给变量加版本号/时间戳。
每次修改版本号+1，检查时同时检查值和版本号。”

大白话理解

- 你看到桌上有一张100元（A），你走开一下，你室友拿走了（变成B），又放回一张100元（变回A）。
- 你回来看，还是100元，就以为没人动过。但钞票可能已经被换过了。
- 解决办法：每张钞票上印一个编号，版本号变了就知道动过。

12. AQS (AbstractQueuedSynchronizer)

标准参考答案

“AQS是JUC同步器的基石，如`ReentrantLock`、`CountDownLatch`。
核心：一个`volatile int state`表示同步状态，一个`CLH`队列管理等待线程。
`ReentrantLock`的实现：

- 非公平锁：线程上来就尝试CAS修改state，成功则获取锁。
- 失败则进入队列，调用 `LockSupport.park()` 阻塞。
- 解锁时唤醒队列头部的线程。”

大白话理解

- AQS像一个排队管理器。
- state是“锁被占了吗？”0=没人占，1=有人占。
- 如果锁被占，后来的线程就排进一个FIFO队列里等着。
- 解锁时，叫醒队列第一个人。

13. 线程池参数

标准参考答案

“核心参数：

- `corePoolSize`：核心线程数，即使空闲也保留
- `maximumPoolSize`：最大线程数
- `keepAliveTime`：非核心线程空闲多久回收

- `workQueue`：任务队列

- `threadFactory`：线程工厂

- `handler`：拒绝策略

规则：当线程数 < core，新建线程；当线程数 ≥ core，任务入队；当队列满且线程数 < max，新建非核心线程；当队列满且线程数 = max，执行拒绝策略。

拒绝策略：Abort（抛异常）、CallerRuns（调用者线程执行）、Discard（丢弃）、DiscardOldest（丢弃队列头）。"

大白话理解

- 核心线程就像正式员工，非核心线程像临时工。
- 来了任务：先让正式员工做。正式员工忙不过来，任务排队。
- 队伍太长（队列满了），招临时工。
- 临时工也满了，就拒绝新任务（抛出异常或自己处理）。

14. ConcurrentHashMap 1.7 vs 1.8

标准参考答案

"1.7: **分段锁**（Segment数组），每个Segment继承ReentrantLock，默认16个Segment，并发度16。

1.8: **CAS + synchronized**，锁粒度更细，只锁链表的头结点或红黑树的根节点。

1.8引入了红黑树（链表长度≥8且数组≥64时转树）。

性能上1.8更好，因为锁冲突更少。"

大白话理解

- 1.7: 把一个大HashMap分成16个小段，每一段各自上锁。两个人操作不同段，可以并行。
- 1.8: 不再分小段，而是锁**单个桶**（一个链表或树）。两个人操作不同桶，完全并行。
- 1.8还优化了链表太长的的问题，超过8个节点变成红黑树，查询更快。

四、JVM

15. 双亲委派模型

标准参考答案

"类加载器收到加载请求时，先委派给父加载器，父加载器再向上委派，直到BootstrapClassLoader。

如果父加载器无法加载，才由子加载器自己尝试。

好处：

1. 避免重复加载（父加载器已加载的类，子不用再加载）
2. 保证核心API不被篡改（例如 `java.lang.Object` 永远由Bootstrap加载，用户自定义的同名类无法覆盖）。"

大白话理解

- 儿子（自定义类加载器）要干活，先找爸爸（扩展类加载器），爸爸再找爷爷（启动类加载器）。
- 爷爷能干的就爷爷干，爷爷干不了，才往下传。

- 这样保证了Java自带的类（如String）不会被你自己写的同名类替换掉。

16. G1 vs CMS

标准参考答案

“CMS（Concurrent Mark Sweep）：

- 特点：低停顿，但会产生内存碎片，且Full GC时停顿大。
- 步骤：初始标记（STW）→ 并发标记 → 重新标记（STW）→ 并发清除。
G1（Garbage First）：

- 特点：将堆分成多个Region，优先回收垃圾最多的Region。
- 可以预测停顿时间（-XX:MaxGCPauseMillis）。
- 不会产生严重碎片，通过**复制算法**整理内存。
- 整体上G1更优，JDK9后默认。”

大白话理解

- CMS：扫地的时候大部分不停止（并发），但扫完地面会有碎渣（碎片），时间久了要停下来大扫除（Full GC）。
- G1：把房间分成多个格子，优先扫垃圾最多的格子。可以控制每次扫的时间（比如最多停200ms）。扫的过程中会把垃圾清走并把有用的东西整理好，不留碎片。

17. 对象在内存中的布局

标准参考答案

“对象在堆内存中的布局分为三部分：

1. **对象头**（Header）：
 - Mark Word：存储哈希码、GC分代年龄、锁状态标志。
 - Klass Pointer：指向类元数据的指针。
2. **实例数据**（Instance Data）：对象中声明的各种字段。
3. **对齐填充**（Padding）：将对象大小补齐为8字节的整数倍。”

大白话理解

- 每个Java对象就像一个小盒子。
- 盒子的最前面贴了一张标签（对象头）：写了这个对象的身份证号（哈希码）、年龄（GC年龄）、有没有被锁等。
- 中间是真正放的东西（你的成员变量）。
- 最后可能加一点泡沫（对齐填充），让盒子大小整齐，方便搬运。

由于篇幅很长，我先把**前17个问题**的标准答案+大白话解释给你。剩下的问题（Spring、MySQL、Redis、RocketMQ、场景设计、非技术问题）我会在接下来的回复中继续补充。你可以先消化这些，然后我接着写后面的。

你要我现在继续写完剩下的所有问题吗？（这样你就有完整的一份面试答案手册）

没问题，以下是**剩余所有问题**的完整答案，从第18题到第38题。每个问题都包含“**标准参考答案**”（面试可直接说）和“**大白话理解**”（帮你搞懂）。你可以直接复制到文档里背诵或练习。

五、Spring 框架

18. Bean生命周期

标准参考答案

Spring Bean 从创建到销毁的完整生命周期：

1. **实例化**：通过构造器或工厂方法创建Bean实例。
2. **属性赋值**：依赖注入（@Autowired、setter方法等）。
3. **初始化前**：调用 BeanPostProcessor 的 `postProcessBeforeInitialization`。
4. **初始化**：执行 `@PostConstruct`、`InitializingBean.afterPropertiesSet()`、自定义 `init-method`。
5. **初始化后**：调用 BeanPostProcessor 的 `postProcessAfterInitialization`（此时生成代理对象，如 AOP）。
6. **使用**：Bean准备就绪。
7. **销毁**：执行 `@PreDestroy`、`DisposableBean.destroy()`、自定义 `destroy-method`。

大白话理解

- 你出生（实例化）→ 给你配玩具（属性赋值）→ 打疫苗/起名字（初始化前后）→ 开始干活（使用）→ 临终遗言（销毁）。
- AOP就是在第5步（初始化后）偷偷把你的方法包装一下，加个日志或权限检查。

19. 循环依赖与三级缓存

标准参考答案

Spring 通过**三级缓存**解决单例Bean的循环依赖：

- 一级缓存：`singletonObjects`，存放完全初始化好的Bean。
 - 二级缓存：`earlySingletonObjects`，存放提前暴露的、未完全初始化的Bean（半成品）。
 - 三级缓存：`singletonFactories`，存放 `ObjectFactory`（可生成代理对象的工厂）。
流程：A依赖B，B依赖A。
1. A实例化后，把自己暴露到三级缓存。
 2. A填充属性时发现需要B，去创建B。
 3. B实例化后，发现需要A，从三级缓存获取A的工厂，生成A的引用（提前暴露），放入二级缓存。
 4. B完成初始化，A再从二级缓存拿到完全初始化的B。
- 为什么需要三级缓存？因为AOP代理对象需要在属性注入前就暴露，三级缓存允许在此时生成代理。

大白话理解

- A和B互相依赖，就像两个人互相等对方先开口。
- 三级缓存相当于一个“预备队”：A刚出生（半成品）就喊“我可以帮忙，但还没学完本事”。
- B需要A时，先拿这个半成品用着，等A学完本事再替换。
- 如果不需要AOP，二级缓存就够了；要AOP，必须三级缓存来提前生成代理。

20. AOP：动态代理 vs CGLIB

标准参考答案

- **JDK动态代理**：要求目标类实现至少一个接口。代理类在运行时创建，使用 `InvocationHandler` 拦截方法调用。
- **CGLIB**：通过**继承**目标类生成子类作为代理，重写父类方法。无法代理 `final` 类或 `final` 方法。
Spring默认：如果目标类有接口，用JDK代理；否则用CGLIB。可配置强制使用CGLIB。
性能：JDK 8以后两者差别不大，CGLIB创建代理对象更慢但调用更快。

大白话理解

- JDK动态代理：你有一个“歌手接口”，实现类“周杰伦”。代理是“替身”，必须也实现歌手接口，假装自己是周杰伦。
- CGLIB：不管你有没有接口，直接生一个“周杰伦的儿子”，继承老爸的所有行为，改掉关键方法。
- 如果周杰伦是 `final` 的（不能有儿子），那CGLIB就没办法了。

六、MySQL

21. B+树索引 vs 哈希索引

标准参考答案

- **B+树索引**：所有数据都存储在叶子节点，叶子节点之间用指针连接形成有序链表。支持**范围查询**和**排序**。多路平衡树，树高较低（3~4层可存上亿数据）。
- **哈希索引**：通过哈希函数计算位置，等值查询极快（ $O(1)$ ），但不支持范围查询，且哈希冲突时性能下降。
InnoDB默认使用B+树，Memory引擎可支持哈希索引。

大白话理解

- B+树像一本**字典**的目录，你能按拼音（范围）找到“从ba到bu”的所有字。
- 哈希像**字典的页码表**，你只能精确查“把”字在第几页，没法查“把”到“吧”之间的字。

22. MVCC 实现原理

标准参考答案

MVCC（多版本并发控制）让读操作不阻塞写操作，提高并发性。

核心：每行记录隐藏两个列——`DB_TRX_ID`（最后修改该行的事务ID）和`DB_ROLL_PTR`（指向undo log中旧版本记录的指针）。

事务启动时生成一个**Read View**，包含：

- `creator_trx_id`: 当前事务ID
 - `m_ids`: 活跃（未提交）的事务ID列表
 - `min_trx_id`: `m_ids`中的最小值
 - `max_trx_id`: 下一个将被分配的事务ID
- 可见性判断:
- 如果行的 `DB_TRX_ID` < `min_trx_id`, 可见。
 - 如果 `DB_TRX_ID` >= `max_trx_id`, 不可见。
 - 如果 `min_trx_id` <= `DB_TRX_ID` < `max_trx_id`, 且 `DB_TRX_ID` 在 `m_ids` 中, 不可见（未提交）；否则可见。
- 读未提交（RU）不生成Read View；读已提交（RC）每次查询都生成新Read View；可重复读（RR）只在第一次查询时生成Read View。

大白话理解

- 每行数据都有一个“修改者ID”和一个“指向旧版本的指针”。
- 事务开始时拍一张快照（Read View），记录下当前谁正在修改数据。
- 读数据时，根据快照判断：这个数据是不是在我之前就已经提交的？如果是，就读；如果不是（比如是别人正在改还没提交的），就顺着指针去找更旧版本。
- RR级别下，快照只拍一次，所以重复读看到的数据一样；RC级别每次都重新拍，所以能看到别人新提交的数据。

23. 慢SQL定位与优化

标准参考答案

定位：

1. 开启MySQL慢查询日志（`slow_query_log`），设置阈值 `long_query_time`（如1秒）。
2. 使用 `EXPLAIN` 分析慢SQL的执行计划。

优化方向：

- **索引失效**：检查 `type`（最好为 `ref` 或 `range`，避免 `ALL` 全表扫描）、`key`（是否用了正确的索引）、`Extra`（如 `Using filesort`、`Using temporary` 表示需要优化）。
- **避免**：函数操作列（`WHERE DATE(create_time)=...`）、隐式类型转换、`LIKE '%xxx'`、`OR` 条件、`!=` 或 `<>`。
- **分页优化**：大偏移量时用子查询或游标（`WHERE id > last_id LIMIT 10`）。
- **SQL重写**：多表JOIN时小表驱动大表，避免 `SELECT *`。
- **库表结构**：垂直拆分、水平分表、适当冗余减少JOIN。

大白话理解

- 慢SQL就像快递员送货太慢。先找出哪一单慢（慢查询日志），然后用 `EXPLAIN` 看他是怎么走的（执行计划）。
- 常见问题：没走捷径（没走索引）、绕远路（全表扫描）、中途需要整理货物（`filesort/temporary`）。
- 优化就是：建索引、改路线（改写SQL）、把大件分开送（分表）。

七、Redis

24. 分布式锁如何保证释放

标准参考答案

使用Redis实现分布式锁的要点：

1. **加锁**：`SET key value NX EX seconds`，value设为唯一标识（如UUID），避免误删别人的锁。
2. **解锁**：使用Lua脚本，先判断value是否匹配，匹配才删除。

```
if redis.call("get",KEYS[1]) == ARGV[1] then
    return redis.call("del",KEYS[1])
else
    return 0
end
```

3. **避免死锁**：设置合理的过期时间（业务执行时间的几倍）。
4. **锁续期**（看门狗）：如果业务未完成，定时延长过期时间。Redisson实现了自动续期。
5. **高可用**：使用RedLock算法（多个独立Redis实例），但大多数场景单实例加主从就够了。

大白话理解

- 锁就是一把带密码的锁（密码=UUID）。你锁上后，只有你能开。
- 锁要设自动弹开时间（过期时间），万一你挂了，锁也会自动释放，不会死锁。
- 开锁时必须先核对密码（value匹配），不能把别人（后来持有锁的线程）的锁给误删了。
- 如果业务没干完锁就快过期了，看门狗会自动帮你续期。

25. RDB vs AOF

标准参考答案

RDB：

- 通过 `save` 或 `bgsave` 生成**全量数据快照**（二进制文件）。
- 优点：文件小，恢复快，适合备份。
- 缺点：可能丢失最后一次快照之后的数据。

AOF：

- 记录每个写命令（类似binlog），追加到文件。
- 优点：数据更完整（可配置 `appendfsync always` 丢失少）。
- 缺点：文件大，恢复慢。

混合持久化（Redis 4.0+）：AOF重写时生成RDB内容+AOF增量，兼顾恢复速度和数据完整性。

大白话理解

- RDB：每周拍一张全家福。系统崩了，只能恢复到上次拍照时的样子，之后的变化全丢。
- AOF：每天写日记，记下每件事。崩了之后，重放日记就能恢复，但日记可能很厚，恢复慢。

- 混合模式：先拍一张全家福，然后记下全家福之后的日记。

26. 哨兵集群选举Leader

标准参考答案

哨兵集群通过**Raft-like**协议选举Leader，过程：

1. 每个哨兵实例向其他哨兵发送 `is-master-down-by-addr` 命令，表示自己认为主库下线。
2. 当**多数哨兵**（超过quorum）同意主库客观下线后，触发故障转移。
3. 候选哨兵（参选者）给自己**增加一个配置纪元**，广播请求投票。
4. 每个哨兵**先到先得**投票，且每个纪元只投一次。
5. 获得**超过半数**投票的哨兵成为Leader。
6. Leader负责从从库中选新主库（优先级、复制偏移量、Run ID等规则），并通知其他从库复制新主库。

大白话理解

- 多个哨兵轮流当班长。
- 如果大家都觉得老师（主库）不在了，就发起投票。
- 谁先喊“我要当班长”，大家就投他（先到先得）。
- 票数超过一半的人当上班长，由他决定谁来当新老师，并通知所有同学。

八、RocketMQ

27. 消息确认机制（ACK）

标准参考答案

RocketMQ采用**自动确认**机制（不同于RabbitMQ的手动ACK）。

- 消费者拉取消息后，会**自动**将消费进度提交到Broker（通过 `offsetStore`）。
- 如果消费者处理失败，可以返回 `ConsumeConcurrentlyStatus.RECONSUME_LATER`，Broker会稍后重试。
- 重试次数超过配置（默认16次）后，消息进入**死信队列**。
- 为保证消息不丢失，生产者可发送同步消息并处理SendResult，消费者建议使用**手动确认模式**（设置 `consumer.setConsumeMessageBatchMaxSize(1)` + 业务处理成功后返回 `CONSUME_SUCCESS`）。

大白话理解

- 普通模式：你从RocketMQ拿走消息，MQ就认为你肯定能处理完，自动帮你记“已消费”。
 - 如果处理失败了，你要主动说“我没搞定，再给我一次机会”。
 - 反复失败多次后，消息被扔进“死信队列”人工处理。
 - 保险做法：处理完业务逻辑（比如数据库写成功）再告诉MQ“我成功了”。
-

28. 死信队列

标准参考答案

进入条件：消息重试次数超过 `maxReconsumeTimes`（默认16次）后，不再投递给消费者，而是转发到死信队列（Dead Letter Queue）。

特点：

- 死信队列名称为 `%DLQ%{consumerGroup}`。
- 消息体不变，但会保留原始队列信息。
- 死信队列中的消息**不会再被消费者自动消费**，需要人工介入或写程序订阅处理。
- 死信队列的消息默认保留3天。

处理方式：人工检查死信原因，修复消费者逻辑后，重新将消息发回原队列或直接落盘。

大白话理解

- 一个快递反复投递16次都失败（比如地址总错），就不再投了，转到“死信仓库”。
- 仓库里的包裹不会自动再送，需要人去看到底什么问题，修好后再重新发货。

29. 顺序消息如何保证全局有序

标准参考答案

RocketMQ保证**分区有序**，不保证全局有序。

实现：

1. 生产者将需要顺序处理的消息发送到**同一个MessageQueue**（通过自定义 `MessageQueueSelector`，例如按订单ID哈希取模）。
2. 消费者端使用**一个线程**消费该队列（`ConsumeOrderlyStatus`），默认一个队列只被一个消费者线程处理。
3. 消费失败时，会**暂停其他消息的消费**，原地重试，避免乱序。
注意：顺序消息的性能较低，且不能使用并发消费。

大白话理解

- 订单A的“下单→支付→发货”三条消息必须按顺序处理。
- 让这三条消息都进入同一个“小格子”（MessageQueue），然后派一个专门的工人（单线程）依次处理这个格子里的消息。
- 如果“支付”消息处理失败了，工人会等它重试成功再处理“发货”，不会跳过去。
- 代价是速度变慢，因为工人一次只能处理一条。

九、场景设计题

30. 弹幕系统设计

标准参考答案

核心需求：高并发写入、实时推送、存储历史弹幕。

设计：

- **写入：**弹幕通过WebSocket或HTTP接入，先写入**消息队列**（如Kafka）削峰，再异步落库（MySQL + 分表）。
- **推送：**使用**WebSocket长连接**，服务端将弹幕广播给房间内的所有用户。为减轻压力，可在客户端做**本地聚合**（如100ms内的弹幕一起展示）。
- **存储：**弹幕按视频ID分片，使用**时序数据库**（如InfluxDB）或Elasticsearch，支持按时间范围查询。
- **过滤：**接入敏感词过滤（DFA算法或第三方API）。
- **扩展：**弹幕展示层（Canvas）与逻辑层分离，支持弹幕速度、密度控制。

大白话理解

- 用户发弹幕：先丢进一个大池子（消息队列），慢慢存到数据库，不直接写库（太慢）。
- 推送弹幕：用户和服务器之间保持电话一直在线（WebSocket），服务器收到弹幕就喊给所有人听。
- 人多了怎么办？同一个房间内几百人，服务器把弹幕发出去后，客户端自己决定每隔0.1秒显示一批，别一条一条闪瞎眼。
- 历史弹幕存ES，方便你拖进度条时加载之前的弹幕。

31. 秒杀系统优化（10w/s）

标准参考答案

分层过滤 + 异步化：

1. **流量前置：**CDN静态化秒杀页面，随机验证码、答题、前端限流（点击一次后按钮灰掉）。
2. **接入层：**Nginx限流（`limit_req`），按IP或UserID限流，拒绝超限请求。
3. **业务层：**
 - 使用Redis预扣库存（Lua脚本原子检查+扣减）。
 - 成功者生成**令牌**，放入消息队列（如RocketMQ），失败者直接返回“已售罄”。
4. **数据库层：**
 - 消费者批量处理消息，异步生成订单，真正扣减DB库存。
 - DB库存使用**乐观锁**或 `update stock set stock = stock - 1 where id = ? and stock >= 1`。
5. **防刷：**IP限流、设备指纹、验证码、令牌桶。
6. **兜底：**熔断降级，返回友好提示。

大白话理解

- 10万人抢100台手机，不能让每个人都冲进数据库。

- 先让Nginx拦住大部分重复请求（一个人一秒最多点一次）。
- 然后Redis快速检查还有没有货，有货就发一个“购买资格券”（令牌），没货直接拒绝。
- 拿到券的人把请求放进消息队列，后端慢慢处理生成订单。
- 这样数据库只处理真正抢到的人，不会被压垮。

32. 分布式ID生成（雪花算法）

标准参考答案

雪花算法（Snowflake）：64位长整型，结构：

- 1位符号位（0）
- 41位时间戳（毫秒级，可用69年）
- 10位工作机器ID（支持1024个节点）
- 12位序列号（每毫秒每节点最多4096个ID）

问题与解决：

- **时钟回拨**：如果机器时间回拨，可能产生重复ID。解决方案：
 - 记录上一次生成ID的时间戳，如果当前时间小于上次时间，等待时间追上或抛出异常。
 - 使用**ZooKeeper**持久化节点版本号作为机器ID的一部分。
 - 百度UidGenerator使用“未来时间补偿”或“环形缓冲区”。
- **节点ID分配**：通过ZK或数据库自动分配。

大白话理解

- 每个ID = 时间戳（谁先谁后） + 机器号（哪台机器） + 序号（同一毫秒内第几个）。
- 问题：如果机器时间不小心往回调了（比如从10:00:01调到10:00:00），后面生成的ID可能比之前的还小，甚至重复。
- 解决办法：发现时间回拨了，就等一等，等到时间追上来再生成；或者直接报错，让运维处理。

33. 限流算法：漏桶 vs 令牌桶

标准参考答案

漏桶：

- 水（请求）以任意速率流入，桶以**固定速率**漏出。
- 桶满了则丢弃请求。
- 特点：**强制平滑突发流量**，输出恒定，适合保护下游系统（如数据库）。

令牌桶：

- 以固定速率向桶中添加令牌，请求需要获取令牌才能通过。
 - 桶容量有限，可以积累令牌应对突发流量。
 - 特点：**允许一定程度的突发**，适合处理有峰值的业务（如秒杀）。
- 实现：Guava `RateLimiter` 是令牌桶，Sentinel 支持两种。

大白话理解

- 漏桶：水龙头出水速度固定。就算你突然倒一盆水，它还是一滴一滴往下流。适合保护脆弱的服务。
- 令牌桶：每秒钟往桶里放10个令牌，你拿一个令牌才能通过一次。如果闲的时候桶里攒了100个令牌，忙的时候可以一口气用完。适合应对突发流量。

十、非技术问题（软技能）

34. 项目中遇到的最大技术难题及解决

标准参考答案

“在镜海视频平台中，我们遇到**大模型QPS限制**的问题。讯飞星火免费版QPS仅为2，无法满足多用户同时使用。我的解决方案：**构建账号池**，存储多个账号的凭证，通过Redis记录每个账号的调用计数和时间窗口，使用Lua脚本保证原子性分配。同时实现降级：当所有账号都达到上限时，提示用户稍后重试。最终我们将整体QPS提升到账号数 $\times 2$ ，并支持动态扩容。从中学到：面对第三方限制，可以通过**资源池化+软负载均衡**来突破瓶颈。”

大白话理解

- 当时最头疼的是大模型一秒只能处理2个请求。
- 我们搞了多个账号，写了个调度器自动切换。
- 虽然每个账号还是2，但账号多了总体就快了。
- 这个经验让我明白，很多限制都可以用“多资源轮换”来破解。

35. 如果重新设计这个项目，会改进哪里？

标准参考答案

“我会在以下方面改进：

1. **可观测性**：增加全链路追踪（如SkyWalking）和更详细的监报告警，比如每个账号的大模型调用耗时和失败率。
2. **弹性伸缩**：将账号池的QPS阈值配置化，接入动态调度，根据实时负载自动增减账号。
3. **消息队列可靠性**：使用RocketMQ的事务消息保证数据同步的最终一致性，避免定时任务补偿的延迟。
4. **测试**：补充压力测试和混沌工程，验证合并请求、断点续传等模块在极端情况下的表现。”

大白话理解

- 当时只顾着实现功能，监控不够细。现在想加一个“指挥中心”，随时看每个账号忙不忙。
 - 另外，数据同步偶尔会有几秒延迟，想用更可靠的事务消息来保证。
 - 还想做更严格的测试，比如故意断网看看断点续传能不能扛住。
-

36. 如何保证代码质量？

标准参考答案

1. **编码规范**：遵循阿里巴巴Java开发手册，使用Checkstyle、SonarQube静态扫描。
2. **单元测试**：核心逻辑覆盖率≥80%，使用JUnit + Mockito。
3. **代码审查**：每次提交至少一人Review，重点关注设计合理性和潜在性能问题。
4. **持续集成**：Git push触发自动化测试、构建、打包，失败则阻止合并。
5. **文档**：复杂模块必须有设计文档和流程图。
6. **自测**：上线前在预发环境进行冒烟测试和性能压测。

大白话理解

- 代码质量就像做菜：要有菜谱（规范）、试吃（单元测试）、别人品鉴（Code Review）、自动化流水线（CI）、写清楚步骤（文档）。
- 避免“我自己写的我自己都看不懂”。

37. 平时怎么学习新技术？

标准参考答案

“我主要通过三种方式：

1. **官方文档 + 源码**：比如学习Spring Boot，先看官方Reference Doc，然后debug关键启动流程。
2. **动手实践**：把学到的技术用在个人项目或开源贡献上，比如我用RocketMQ实现了项目中的异步削峰。
3. **高质量社区**：关注技术博客（如美团技术团队、阿里中间件）、B站优质UP主（如狂神说、编程不良人），也经常看Stack Overflow和GitHub Issues。
我会带着问题去学，比如‘为什么Redis用单线程模型还那么快？’然后查资料、做实验、总结成笔记。”

大白话理解

- 看官方手册是最权威的，看不懂再搜视频。
- 光看不动手等于白学，所以我一定写demo跑起来。
- 遇到问题就去Stack Overflow或GitHub issues找答案，顺便记笔记。

38. 你希望从实习中获得什么？

标准参考答案

“我希望获得三方面成长：

1. **工程能力**：参与真实的高并发项目，学习大厂的代码规范、发布流程、监控运维体系。
2. **团队协作**：学会如何在多人协作下高效沟通、使用Git进行分支管理、参与技术评审。
3. **职业素养**：了解业务驱动开发的方式，培养对线上服务的敬畏心（如变更管控、灰度发布、回滚预案）。
同时我也希望能用自己的技术为团队做出实际贡献，不只是一个学习者。”

大白话理解

- 我想看看真正的工业级代码是怎么写的，不是学校那种小demo。
 - 想学会怎么和产品、测试、其他开发配合，别自己闷头写。
 - 想体验“上线出bug”的紧张感，学会怎么避免和快速修复。
 - 当然我也会努力干活，不是来混日子的。
-

全部38个问题已回答完毕。

建议你按照以下步骤使用：

1. **先理解大白话**，确保自己真的懂了这个概念。
2. **背标准答案的关键词**（每个答案里都有几个重点短语）。
3. **用自己的话复述**，录音听一下是否自然。
4. **找人模拟面试**，让对方随便抽问题问你。

祝你面试顺利！如果需要我把这些内容整理成 **PDF** 或 **Markdown文件** 方便打印背诵，也可以告诉我。